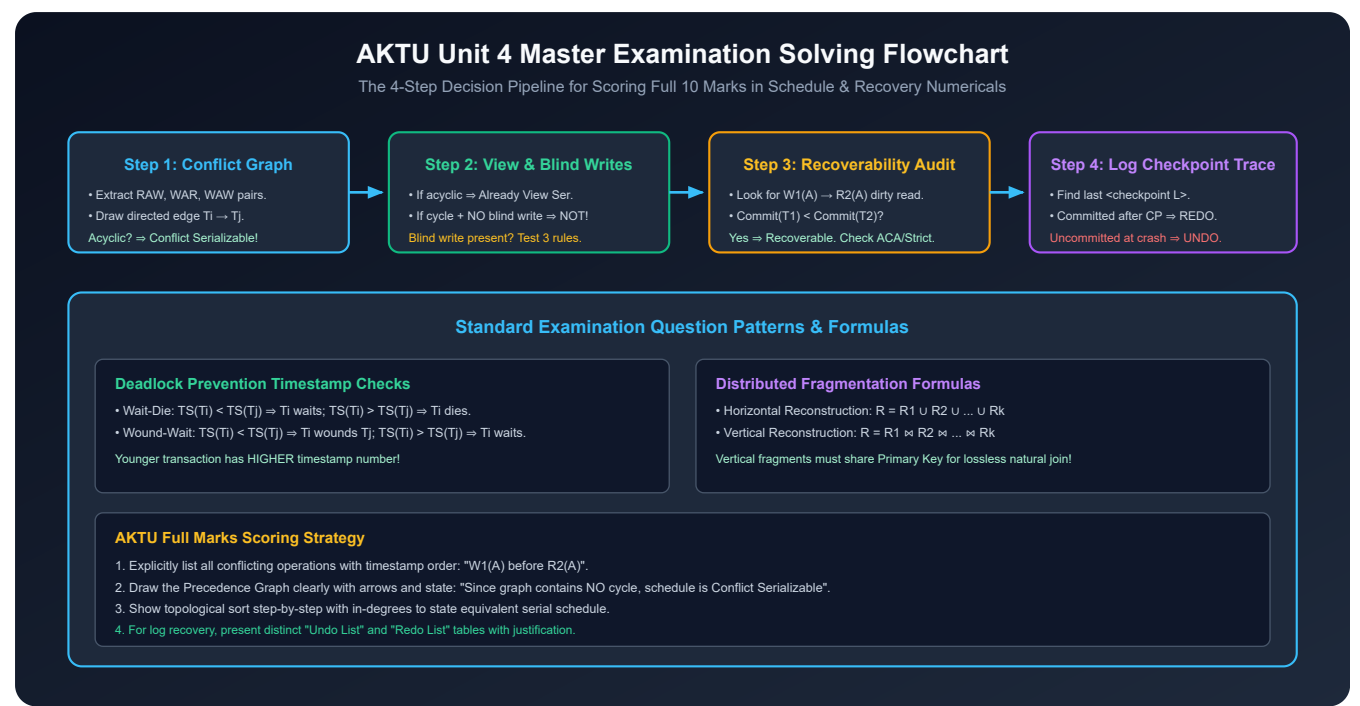


Module 13: AKTU PYQs & Solved Schedules



- On $W_3(Y)$ is followed by $W_2(Y) \implies \mathbf{T_3} \rightarrow \mathbf{T_2}$ (Already present).
- On $R_1(Z)$ is followed by $W_2(Z) \implies \mathbf{T_1} \rightarrow \mathbf{T_2}$.

Precedence Graph Edges:

$\mathbf{T_3}$

$\rightarrow \mathbf{T_1}, \quad \mathbf{T_3}$

$\rightarrow \mathbf{T_2}, \quad \mathbf{T_1}$

$\rightarrow \mathbf{T_2}$

Cycle Check: No cycles exist in the graph (strictly Acyclic Directed Graph).

Topological Sorting:

- In-degree of $\mathbf{T_3} = 0 \implies \mathbf{T_3}$ is first.
- In-degree of $\mathbf{T_1} = 0$ (after removing $\mathbf{T_3}$) $\implies \mathbf{T_1}$ is second.
- Remaining: $\mathbf{T_2}$ is third.

Equivalent Serial Schedule: $\mathbf{T_3}$

$\rightarrow \mathbf{T_1}$

$\rightarrow \mathbf{T_2}$.

Verdict 1: Schedule SS is **Conflict Serializable!**

2. Recoverability Test

Look for Dirty Reads (W_i

$\rightarrow R_j$):

- $W_3(Y)$ is followed by $R_2(Y)$ while $\mathbf{T_3}$ is uncommitted.
- So $\mathbf{T_2}$ reads uncommitted data from $\mathbf{T_3}$.

Check commit orders:

- $\mathbf{T_2}$ commits at step C_2 .
- $\mathbf{T_3}$ commits at step C_3 .
- Notice: $C_2 < C_3$! That means **$\mathbf{T_2}$ COMMITS BEFORE $\mathbf{T_3}$ COMMITS!**

If $\mathbf{T_3}$ were to abort, $\mathbf{T_2}$ cannot be rolled back!

Verdict 2: Schedule SS is **IRRECOVERABLE!**

3. Cascadeless Test

In a cascadeless schedule, no transaction can read uncommitted data.

Since T_2 reads Y from T_3 before T_3 commits, dirty read exists.

Verdict 3: Schedule SS is **NOT Cascadeless!**

2. AKTU PYQ 2 (10 Marks — 2021-22 End-Sem)

Question:

Explain Log-Based Recovery with Immediate Database Modification. Suppose a system crashes with the following log contents:

,
,

What actions will the recovery manager take during crash recovery? State the Undo-List and Redo-List.

Solution:

1. Locate Checkpoint:

- Checkpoint is $\langle \text{checkpoint} \rangle \{T_1, T_2\}$

$\angle$.

- Active transactions at checkpoint: $\{T_1, T_2\}$.

2. Initialize Lists:

- $\text{Undo-List} = \{T_1, T_2\}$

- $\text{Redo-List} = \emptyset$

3. Forward Scan from Checkpoint to Crash:

- $\langle T_3 \text{ start} \rangle$

$\angle \implies$ Add T_3 to Undo-List: $\text{Undo-List} = \{T_1, T_2, T_3\}$.

- $\langle T_1 \text{ commit} \rangle$

$\angle \implies$ Move T_1 to Redo-List: $\text{Redo-List} = \{T_1\}$, $\text{Undo-List} = \{T_2, T_3\}$.

- $\langle T_2 \text{ commit} \rangle$

$\angle \implies$ Move T_2 to Redo-List: $\text{Redo-List} = \{T_1, T_2\}$, $\text{Undo-List} = \{T_3\}$.

4. Final Lists at Crash Point:

- $\text{Redo-List} = \{T_1, T_2\}$ (Both started and committed).

- $\text{Undo-List} = \{T_3\}$ (Started, updated D , but never committed before crash).

5. Recovery Actions Performed:

- **Undo Phase:** Rollback T_3 : Restore data item $D = 400$ (its old value), write $\langle T_3 \text{ abort} \rangle$

angle\$.

- **Redo Phase:** Replay T_1 and T_2 : Set $A = 100$, $B = 250$, $C = 350$.

3. Architectural Diagram
